



School of Mathematics and Computer Science

[Final Exam]

Fall 2022: CSE 309: Theory of Automata

Instructors: Shahid Hussain, Asma Larik

Sunday, December 18, 2022. Duration: 180 minutes

— SOLUTION —

Instructions

- Use of mobile devices is strictly prohibited for the entirety of the exam period.
- Please submit your devices in your bag at the front of the examination room.
- You may keep writing material with you on your desk.
- Please **do not** use a pencil or a red pen for answers that you want to be graded.
- Please write your answers in the **provided space**.
- Please **do not** tear/remove any page.
- Acquisition of answers through unfair means will automatically cancel your exam.
- Keep track of the time.
- Please write your **name** and **student ID** clearly on all pages you submit.
- This exam contains **7 questions** (including a bonus question) for a total of **35 points plus 5 bonus points**.
- This exam consists of **total of 5 sides of printed pages** including this one.

Question 1

[10 points]

Let L_1 and L_2 be two Turing-recognizable languages (recursively enumerable languages). Show that the class of Turing-recognizable languages is closed under:

- (a) (5 points) The operation of union i.e., $L_1 \cup L_2$ is also Turing-recognizable.

Solution

Let M_1 and M_2 be two TM's that recognize the languages L_1 and L_2 , respectively. Now we can construct a new TM M which recognizes $L_1 \cup L_2$ as following:

M = "On input w :

1. Pass w to both M_1 as well as M_2
2. If any of the machines M_1 or M_2 accept, *accept*."

- (b) (5 points) The operation of concatenation i.e., $L_1 L_2$ is also Turing-recognizable.

Solution

Let M_1 and M_2 be two TM's that recognize the languages L_1 and L_2 , respectively. Now we can construct a new NTM N which recognizes $L_1 L_2$ as following:

N = "On input w :

1. Nondeterministically, break w as $w_1 w_2$
2. Pass w_1 to M_1 and w_2 to M_2
3. If both M_1 and M_2 accept, *accept*."

Question 2

[5 points]

If A is an infinite Turing-recognizable language and $B \subseteq A$. Is B necessarily a Turing-recognizable language? (Prove or disprove.)

Solution

No.

Counterexample: Let $A = \Sigma^*$ and $B = \overline{A_{\text{TM}}}$. Clearly $B \subseteq A$. A is regular, hence Turing-recognizable however B is not Turing-recognizable (see Question 5).

Question 3

[5 points]

Prove the following statement:

If A is **NP**-complete and $A \leq_{\mathbf{P}} B$ for some problem $B \in \mathbf{P}$ then $\mathbf{P} = \mathbf{NP}$.

Solution

If A is an **NP**-complete language and A reduces to some language B in **P**. Then it means every language $L \in \mathbf{NP}$ reduces to B (by definition of **NP**-completeness). Implying, every language in **NP** (including **NP**-complete) can be decided by some deterministic Turing machines in polynomial time, which in turn implies $\mathbf{P} = \mathbf{NP}$.

Question 4

[5 points]

A subset of the nodes of a simple undirected graph G is called a *dominating set* if every other node of G is adjacent to some node in the subset.

$$D = \{\langle G, k \rangle : G \text{ has a dominating set of } k \text{ nodes}\}.$$

Show that the language D (the language of dominating set) is in class **NP**.

Solution

Following algorithm will verify an instance $w = \langle G, k \rangle$ in polynomial time if $w \in D$ or not.

Let the graph $G = (V, E)$ be an undirected graph and w be a subset of V of cardinality k . For each vertex $v \in V \setminus w$. Check if there exists an edge such that one end of this edge is v and the other end is in w . If it holds for all vertices then $w \in D$ otherwise $w \notin D$.

Above algorithm can verify in polynomial time if a given instance of the problem is in D or not.

Question 5

[5 points]

Show that the language $\overline{A_{\mathbf{TM}}}$ is not Turing-recognizable.

Solution

We know that $A_{\mathbf{TM}}$ is Turing-recognizable. If $\overline{A_{\mathbf{TM}}}$ were also Turing-recognizable, then $A_{\mathbf{TM}}$ would be decidable. We know $A_{\mathbf{TM}}$ is not decidable, so $\overline{A_{\mathbf{TM}}}$ must not be Turing-recognizable.

Question 6

[10 points]

State whether following statements are **True** or **False** ($x = 1$ point for correct answer). Justify your answers as well ($x \times 1$ point for correct justification).

- (a) If L_1 is a Turing-decidable language and L_2 is a Turing-recognizable language then $L_1 - L_2$ is a Turing-recognizable language.

Solution

FALSE

Justification: The set of Turing-recognizable languages is not closed under set difference. Therefore, $L_1 - L_2$ is not necessarily a Turing-recognizable.

- (b) The problem CLIQUE is in the class **NP**.

Solution

TRUE

Justification: Given a set of vertices U (a certificate) as a clique. We can verify if all vertices in U are connected by simple polynomial time algorithm (verification algorithm). Therefore, the problem CLIQUE is in the class **NP**.

- (c) The complement of every Turing-decidable language is Turing-decidable.

Solution

TRUE

Justification: Let L be a Turing-decidable language and M a TM that decides L . We can use M to construct a new TM M' that decides $\bar{L}$ as following:

$M' =$ “On input w :

1. Pass w to M .
2. If M accepts, *reject*.
3. If M rejects, *accept*.”

Clearly M' decides $\bar{L}$.

- (d) There exists a Turing machine that recognizes A_{TM} .

Solution

TRUE

Justification: We can use following Turing machine U that recognizes A_{TM} .

$U =$ “On input $\langle M, w \rangle$:

1. Simulate M on input w .
2. If M ever enters its accept state, *accept*; if M ever enters its reject state, *reject*.”

- (e) The problem SAT is in the class **P**.

Solution

FALSE

Justification: Since $\mathbf{P} = \mathbf{NP}$ is still an open problem and it is well known that SAT is an **NP**-complete language therefore SAT is not **P**.

7. (5 points (bonus)) Define a *coding* **Code** to be a mapping, $\text{Code} : \Sigma^* \rightarrow \Sigma^*$ (not necessarily one-to-one).

For some string x , $x = \sigma_1 \cdots \sigma_n \in \Sigma^*$, we define $\text{Code}(x) = \text{Code}(\sigma_1) \cdots \text{Code}(\sigma_n)$ and for a language $L \subseteq \Sigma^*$, we define $\text{Code}(L) = \{\text{Code}(x) : x \in L\}$.

Show that the class **NP** is closed under *coding* that is if a language $L \in \mathbf{NP}$ then $\text{Code}(L) \in \mathbf{NP}$.

Solution

Let N be a NTM that decides L where L is a language in **NP**. Let $|\Sigma| = k$. The coding function **Code** is well-defined.

Let N' be an NTM that decides $\text{Code}(L)$. We can describe N' as following:

$N' =$ "On input $w = x_1x_2 \cdots x_m$:

1. Let y be an empty string
2. Repeat following steps: for $i = 1$ to m
3. Nondeterministically choose l symbols $\alpha_i = x_i x_{i+1} \cdots x_{i+l}$ and find which symbol in Σ map to α_i .
4. Let $y = y\alpha_i$
5. Repeat from Step 2 till symbols in w have been transformed
6. Now pass y to M
7. If M accepts, *accept*."

We can make sure that Step 3 runs in polynomial time as we know there are k symbols in the alphabet.